

Agentic RAG llama GROQ

RAG : Retrieval Augmented Generation

Langchain-groq

Step-1: Install the packages

```
!pip install langchain langchain-groq langchain-community
```

```
Requirement already satisfied: langchain in /usr/local/lib/python3.12/dist-packages (0.3.27)
Collecting langchain-groq
  Downloading langchain_groq-0.3.7-py3-none-any.whl.metadata (2.6 kB)
Collecting langchain-community
  Downloading langchain_community-0.3.29-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: langchain-core<1.0.0,>=0.3.72 in /usr/local/lib/python3.12/dist-packages (from langchain) (0.3.75)
Requirement already satisfied: langchain-text-splitters<1.0.0,>=0.3.9 in /usr/local/lib/python3.12/dist-packages (from langchain) (0.3.10)
Requirement already satisfied: langsmith<0.1.17 in /usr/local/lib/python3.12/dist-packages (from langchain) (0.1.17)
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/lib/python3.12/dist-packages (from langchain) (2.11.7)
Requirement already satisfied: SQLAlchemy<3.0.0,>=1.4 in /usr/local/lib/python3.12/dist-packages (from langchain) (2.0.43)
Requirement already satisfied: requests<3,>=2 in /usr/local/lib/python3.12/dist-packages (from langchain) (2.32.4)
Requirement already satisfied: PyYAML<=5.3 in /usr/local/lib/python3.12/dist-packages (from langchain) (6.0.2)
Collecting groq<1,>=0.30.0 (from langchain-groq)
  Downloading groq-0.31.1-py3-none-any.whl.metadata (16 kB)
Collecting requests<3,>=2 (from langchain)
  Downloading requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Requirement already satisfied: aiohttp<4.0.0,>=3.8.3 in /usr/local/lib/python3.12/dist-packages (from langchain-community) (3.12.1)
Requirement already satisfied: tenacity!=8.4.0,<10,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from langchain-community) (9.0.0)
Collecting dataclasses-json<0.7,>=0.6.7 (from langchain-community)
  Downloading dataclasses_json-0.6.7-py3-none-any.whl.metadata (25 kB)
Requirement already satisfied: pydantic-settings<3.0.0,>=2.10.1 in /usr/local/lib/python3.12/dist-packages (from langchain-community) (2.10.1)
Requirement already satisfied: httpx-sse<1.0.0,>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from langchain-community) (0.4.0)
Requirement already satisfied: numpy>=1.26.2 in /usr/local/lib/python3.12/dist-packages (from langchain-community) (2.0.2)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain-community) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain-community) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain-community) (25.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain-community) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain-community) (6.1.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain-community) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3.8.3->langchain-community) (1.17.0)
Collecting marshmallow<4.0.0,>=3.18.0 (from dataclasses-json<0.7,>=0.6.7->langchain-community)
  Downloading marshmallow-3.26.1-py3-none-any.whl.metadata (7.3 kB)
Collecting typing-inspect<1,>=0.4.0 (from dataclasses-json<0.7,>=0.6.7->langchain-community)
  Downloading typing_inspect-0.9.0-py3-none-any.whl.metadata (1.5 kB)
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from groq<1,>=0.30.0->langchain-groq) (4.6.2)
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from groq<1,>=0.30.0->langchain-groq) (1.9.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from groq<1,>=0.30.0->langchain-groq) (0.23.0)
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from groq<1,>=0.30.0->langchain-groq) (1.3.1)
Requirement already satisfied: typing-extensions<5,>=4.10 in /usr/local/lib/python3.12/dist-packages (from groq<1,>=0.30.0->langchain-groq) (4.12.0)
Requirement already satisfied: jsonpatch<2.0,>=1.33 in /usr/local/lib/python3.12/dist-packages (from langchain-core<1.0.0,>=0.3.72->langchain) (1.33)
Requirement already satisfied: packaging>=23.2 in /usr/local/lib/python3.12/dist-packages (from langchain-core<1.0.0,>=0.3.72->langchain) (25.0)
Requirement already satisfied: orjson>=3.9.14 in /usr/local/lib/python3.12/dist-packages (from langsmith<0.1.17->langchain) (3.11.0)
Requirement already satisfied: requests-toolbelt>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<0.1.17->langchain) (1.0.0)
Requirement already satisfied: zstandard>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<0.1.17->langchain) (0.23.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.7.4->langchain) (0.7.0)
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.7.4->langchain) (2.33.2)
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.7.4->langchain) (0.4.0)
Requirement already satisfied: python-dotenv>=0.21.0 in /usr/local/lib/python3.12/dist-packages (from pydantic-settings<3.0.0,>=2.10.1->langchain) (1.0.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2->langchain) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2->langchain) (3.10.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2->langchain) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2->langchain) (2025.1.1)
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from SQLAlchemy<3,>=1.4->langchain) (3.1.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->groq<1,>=0.30.0->langchain-groq) (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->groq<1,>=0.30.0->langchain-groq) (0.14.0)
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.12/dist-packages (from jsonpatch<2.0,>=1.33->langchain-community) (3.0.0)
Collecting mypy_extensions>=0.3.0 (from typing-inspect<1,>=0.4.0->dataclasses-json<0.7,>=0.6.7->langchain-community)
  Downloading mypy_extensions-1.1.0-py3-none-any.whl.metadata (1.1 kB)
```

```
pip install pypdf
```

```
Collecting pypdf
  Downloading pypdf-6.0.0-py3-none-any.whl.metadata (7.1 kB)
  Downloading pypdf-6.0.0-py3-none-any.whl (310 kB)
310.5/310.5 kB 7.7 MB/s eta 0:00:00
Installing collected packages: pypdf
Successfully installed pypdf-6.0.0
```

```
pip install chromadb
```

```
Collecting chromadb
  Downloading chromadb-1.0.20-cp39-abi3-manylinux_2_17_x86_64_manylinux2014_x86_64.whl.metadata (7.3 kB)
Requirement already satisfied: build>=1.0.3 in /usr/local/lib/python3.12/dist-packages (from chromadb) (1.3.0)
Requirement already satisfied: pydantic>=1.9 in /usr/local/lib/python3.12/dist-packages (from chromadb) (2.11.7)
Collecting pybase64>=1.4.1 (from chromadb)
```

```

Downloading pybase64-1.4.2-cp312-cp312-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_64.whl.metadata
Requirement already satisfied: uvicorn>=0.18.3 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.18.3->chromadb)
Requirement already satisfied: numpy>=1.22.5 in /usr/local/lib/python3.12/dist-packages (from chromadb) (2.0.2)
Collecting posthog<6.0.0,>=2.4.0 (from chromadb)
  Downloading posthog-5.4.0-py3-none-any.whl.metadata (5.7 kB)
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (4.15.0)
Collecting onnxruntime>=1.14.1 (from chromadb)
  Downloading onnxruntime-1.22.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (4.9 kB)
Requirement already satisfied: opentelemetry-api>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (1.36.0)
Collecting opentelemetry-exporter-otlp-proto-grpc>=1.2.0 (from chromadb)
  Downloading opentelemetry-exporter-otlp-proto-grpc-1.36.0-py3-none-any.whl.metadata (2.4 kB)
Requirement already satisfied: opentelemetry-sdk>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (1.36.0)
Requirement already satisfied: tokenizers>=0.13.2 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.22.0)
Collecting pypika>=0.48.9 (from chromadb)
  Downloading PyPika-0.48.9.tar.gz (67 kB)
67.3/67.3 kB 3.7 MB/s eta 0:00:00

Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: tqdm>=4.65.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (4.67.1)
Requirement already satisfied: overrides>=7.3.1 in /usr/local/lib/python3.12/dist-packages (from chromadb) (7.7.0)
Requirement already satisfied: importlib-resources in /usr/local/lib/python3.12/dist-packages (from chromadb) (6.5.2)
Requirement already satisfied: grpcio>=1.58.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (1.74.0)
Collecting bcrypt>=4.0.1 (from chromadb)
  Downloading bcrypt-4.3.0-cp39-abi3-manylinux_2_34_x86_64.whl.metadata (10 kB)
Requirement already satisfied: typer>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.17.3)
Collecting kubernetes>=28.1.0 (from chromadb)
  Downloading kubernetes-33.1.0-py2.py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: tenacity>=8.2.3 in /usr/local/lib/python3.12/dist-packages (from chromadb) (8.5.0)
Requirement already satisfied: pyyaml>=6.0.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (6.0.2)
Collecting mmh3>=4.0.1 (from chromadb)
  Downloading mmh3-5.2.0-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (14 kB)
Requirement already satisfied: orjson>=3.9.12 in /usr/local/lib/python3.12/dist-packages (from chromadb) (3.11.3)
Requirement already satisfied: httpx>=0.27.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.28.1)
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (13.9.4)
Requirement already satisfied: jsonschema>=4.19.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (4.25.1)
Requirement already satisfied: packaging>=19.1 in /usr/local/lib/python3.12/dist-packages (from build>=1.0.3->chromadb) (25.0)
Requirement already satisfied: pyproject_hooks in /usr/local/lib/python3.12/dist-packages (from build>=1.0.3->chromadb) (1.2.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (4.10.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (2025.8.3)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (3.10)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx>=0.27.0->chromadb) (0.14.0)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.0->chromadb) (25.3.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.0->chromadb) (2023.12.1)
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.0->chromadb) (0.35.1)
Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.0->chromadb) (0.22.0)
Requirement already satisfied: six>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (1.17.0)
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (2.9.0.post0)
Requirement already satisfied: google-auth>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (2.37.0)
Requirement already satisfied: websocket-client!=0.40.0,!<0.41.*,!<0.42.*,>=0.32.0 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (1.7.0)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (2.32.5)
Requirement already satisfied: requests-authlib in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (1.3.1)

```

Step-2: Retrive LLM model using groq

```

from langchain_groq import ChatGroq
groq_api="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" # Groq API Key - https://console.groq.com/keys

llm = ChatGroq(
    api_key=groq_api,
    model="llama-3.3-70b-versatile",
    temperature=0.7,
    max_tokens=512
)
llm

ChatGroq(client=<groq.resources.chat.completions.Completions object at 0x78398d186960>, async_client=
<groq.resources.chat.completions.AsyncCompletions object at 0x78398c4e99a0>, model_name='llama-3.3-70b-versatile', model_kwargs={},
groq_api_key=SecretStr('*****'), max_tokens=512)

```

Step-3: Load the pdf

```

from langchain.document_loaders import PyPDFLoader
pdf_path = "/content/nareshiT_context2.pdf"
loader = PyPDFLoader(pdf_path)
documents = loader.load()

```

Step-4: Split into Text

```

from langchain.text_splitter import RecursiveCharacterTextSplitter
# ----- SPLIT -----
splitter = RecursiveCharacterTextSplitter(

```

```

chunk_size=800, # Each split size
chunk_overlap=200 # 200 characters Overlap
)
docs = splitter.split_documents(documents)
print(docs)

```

 [Document(metadata={'producer': 'www.ilovepdf.com', 'creator': 'Microsoft® Word 2016', 'creationdate': '2025-09-06T16:07:13+00:00'},

Step-5: Divide text into Embeddings and store in vector database

```

# ----- EMBEDDINGS + VECTORSTORE -----
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma # Chroma Database
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

vectordb = Chroma.from_documents(
    documents=docs, # PDF
    embedding=embeddings, # Model
)
vectordb.persist()

```

retriever = vectordb.as_retriever(search_kwargs={"k": 10}) # 4 top chunks db : Database Everytime we need to get 10 Outputs

 /tmp/ipython-input-502131884.py:4: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain 0.2.2

```

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as :
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 23.0kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 8.67kB/s]

README.md: 10.5k/? [00:00<00:00, 701kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 2.85kB/s]

config.json: 100% 612/612 [00:00<00:00, 66.4kB/s]

model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 68.9MB/s]

tokenizer_config.json: 100% 350/350 [00:00<00:00, 33.3kB/s]

vocab.txt: 232k/? [00:00<00:00, 8.35MB/s]

tokenizer.json: 466k/? [00:00<00:00, 19.9MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 7.34kB/s]

config.json: 100% 190/190 [00:00<00:00, 12.7kB/s]
/tmp/ipython-input-502131884.py:10: LangChainDeprecationWarning: Since Chroma 0.4.x the manual persistence method is no longer supported
  vectordb.persist()

```

Step-6: Develop a Prompt

```

# ----- CUSTOM PROMPT -----
system_prompt = """
You are a helpful **College Administrator at Naresh IT**.
Your role is to warmly explain training programs, trainers, schedules, placements, and facilities
to new students who are joining the institute.

Context rules:
- Use only the provided CONTEXT chunks from the retriever.
- Explain in a clear, welcoming, and professional tone.
- Highlight key details like trainer names, course schedules, fees, infrastructure, and placement support.
- If multiple trainers or batches are mentioned, clarify them politely.
- Do not make up information beyond the context.
- When useful, mention the chunk/page metadata as your reference.
"""

```

Step-7: Create a PromptTemplate

```

from langchain.prompts import PromptTemplate
from langchain.prompts import PromptTemplate

prompt_template = PromptTemplate(
    template="""

```

You are a helpful **College Administrator at Naresh IT**.
Your role is to warmly explain training programs, trainers, schedules, placements, and facilities to new students who are joining the institute.

Context rules:

- Use only the provided CONTEXT chunks from the retriever.
- Explain in a clear, welcoming, and professional tone.
- Highlight key details like trainer names, course schedules, fees, infrastructure, and placement support.
- If multiple trainers or batches are mentioned, clarify them politely.
- Do not make up information beyond the context.
- When useful, mention the chunk/page metadata as your reference.

CONTEXT:

{context}

QUESTION:

{question}

Answer as the College Administrator.

```
"""  
    input_variables=["context", "question"]  
)
```

Step-8: Build Retrieval QA

```
from langchain.chains import RetrievalQA  
retrieval_qa = RetrievalQA.from_chain_type(  
    llm=llm,  
    retriever=retriever, # Retriever : Vector Database ; Vector Database gives 10 answers  
    chain_type="stuff", # Stuff : Combine all  
    chain_type_kwargs={"prompt": prompt_template}  
)  
# LLM model will get 10 answers from database and combine based on prompt
```

Step-9: Ask the Question

```
query = "Provide me omkar sir class zoom link"  
response = retrieval_qa.run(query)
```

```
print("\n--- Model Answer ---\n")  
print(response)
```

```
🔄 /tmp/ipython-input-3284936125.py:2: LangChainDeprecationWarning: The method `Chain.run` was deprecated in langchain 0.1.0 and will be removed in 0.2.0. Use `chain.invoke` instead.  
    response = retrieval_qa.run(query)
```

```
--- Model Answer ---
```

Welcome to Naresh IT. I'm happy to help you with the Zoom link for Omkar Sir's class. According to our records, the Zoom link for his class is [https://nareshit.zoom.us/j/9876543210](#).

```
query = "Provide me omkar sir reviews"  
response = retrieval_qa.run(query)
```

```
print("\n--- Model Answer ---\n")  
print(response)
```

```
🔄 --- Model Answer ---
```

Welcome to Naresh IT. I'm delighted to share with you the reviews of our esteemed trainer, Omkar Sir. As per our records (based on the latest data), here are a few reviews from our students:

- * Amit Yadav (6 months ago) - "I attended Full Stack Data Science under Omkar Sir. His teaching style is fabulous, very student-friendly, and he makes complex topics easy to understand."
- * Akshay Ghatage (1 week ago) - "Learning from Omkar Sir added great value to my journey in Data Science and AI. His teaching style is clear, concise, and practical."
- * Nagesh Kure (6 months ago) - "I had the privilege of learning Full Stack Data Science & AI from Omkar Sir. He simplifies complex concepts and provides real-world examples."
- * Dodda Sairam (1 week ago) - "Omkar Sir's training in Data Science is excellent. His clear explanations, practical examples, and hands-on approach helped me grasp the concepts quickly."
- * Kanhu Charan Nahak (10 months ago) - "I recently completed the Data Science course at NareshIT under Omkar Sir's guidance, and I highly recommend it to anyone interested in the field."

These reviews demonstrate Omkar Sir's exceptional teaching skills, his ability to make complex concepts easy to understand, and his commitment to providing high-quality training.

```
query = "What are the latest python classes timings"  
response = retrieval_qa.run(query)
```

```
print("\n--- Model Answer ---\n")  
print(response)
```



--- Model Answer ---

Welcome to Naresh IT. I'm happy to help you with your query. Unfortunately, I don't have the most up-to-date information on the latest course offerings. However, I can suggest that you may be interested in our Full Stack Data Science & AI course, which includes Python basics as part of the curriculum. If you're looking for a specific Python course, I recommend checking our website or contacting us directly for the latest information. Please note that the information I provided is based on the context available to me, and I may not have the most current details. I

Its giving information only from document this is Standard RAG approach

If we connect any API calls then fetch the data this indicates AGENTIC Behaviour

✓ Agentic RAG

Step-10: Fetch the data from url

```
from bs4 import BeautifulSoup # Web Scrapping
import requests

course_schedule_url = "https://nareshit.in/course-schedule/"
def fetch_trainer_from_web(query: str):
    """
    Fetch ALL trainer/course details from NareshIT course-schedule page.
    Input: trainer name or course name as plain text.
    Returns: A student-friendly description string with all matches.
    """
    resp = requests.get(course_schedule_url, timeout=10)
    if resp.status_code != 200:
        return f"Could not reach schedule page (status code {resp.status_code})."

    soup = BeautifulSoup(resp.text, "html.parser")
    rows = soup.find_all("tr")

    matches = []
    for row in rows:
        cols = [td.get_text(strip=True) for td in row.find_all("td")]
        if any(query.lower() in c.lower() for c in cols):
            course = cols[0] if len(cols)>0 else "Unknown"
            trainer = cols[1] if len(cols)>1 else "Unknown"
            date = cols[2] if len(cols)>2 else "Unknown"
            time = cols[3] if len(cols)>3 else "Unknown"
            matches.append(
                f"Course: {course}\n"
                f"Trainer: {trainer}\n"
                f"Start Date: {date}\n"
                f"Time: {time}\n"
            )

    if not matches:
        return f"No matching course or trainer found for '{query}' on NareshIT schedule."
    return "\n---\n".join(matches)
```

Step-11: Course Schedule and Trainer Fetcher Tool Definition

```
from langchain.agents import initialize_agent, Tool, AgentType

trainer_tool = Tool(
    name="Course Schedule Web Fetcher",
    func=fetch_trainer_from_web,
    description=(
        "Use this tool to answer queries about courses, trainers, or session schedules "
        "at Naresh IT. Input can be a course name (like 'Python') or trainer name."
    )
)
```

Step-12: Defining a Document Retrieval Tool

```
retriever_tool = Tool(
    name="Document Retriever",
    func=lambda q: retrieval_qa.run(q), # Database Name : retrieval_qa
    description="Answer queries using NareshIT training documents."
)
```

Step-13: Initialise the agent

```
tools = [retriever_tool, trainer_tool]

# Initialize agent with a more flexible agent type
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.CHAT_ZERO_SHOT_REACT_DESCRIPTION, # works with plain text tools ; Zero Shot : means NO training / NO examples
    verbose=True
)
```

Step-14: Provide the Instructions to agent

```
system_instruction = """
You are a helpful College Administrator at Naresh IT.
- If the student asks about sessions, trainers, or course schedules, first try the Document Retriever tool (PDF).
- If no relevant info is found, use the Course Schedule Web Fetcher tool.
- Always provide a friendly, student-facing final answer.
"""
```

Step-15: Start Query

```
query = "Tell me about upcoming session on python."
response = agent.run(f"{system_instruction}\n\nQuestion: {query}")

print("\n--- Admin Answer ---\n", response)
```



```
> Entering new AgentExecutor chain...
Thought: The student is asking about an upcoming session on Python, so I should first try to find the information in the documents

Action:
...
{
  "action": "Document Retriever",
  "action_input": "Python session schedule"
}
...

Observation: Welcome to Naresh IT. I'd be happy to help you with the Python session schedule. Unfortunately, the provided context
If you're Looking for a specific schedule, I can suggest checking our course catalog or reaching out to our admissions team for mo
Additionally, I can inform you about our Full Stack Data Science & AI course, which includes Python basics and is taught by Mr. Om
Please Let me know if there's anything else I can help you with or if you'd Like more information on our courses and schedules.
Thought:Thought: The Document Retriever tool provided some general information about Python-related courses and trainers, but it d

Action:
...
{
  "action": "Course Schedule Web Fetcher",
  "action_input": "Python"
}
...

Observation: Course: Full Stack PYTHON
Trainer: Mr. K V Rao
Start Date: 8th Sep
Time: 6:00 PM

---
Course: PYTHON
Trainer: Mr. K V Rao
Start Date: 8th Sep
Time: 6:00 PM

---
Course: Full Stack PYTHON
Trainer: Mr. Sai Vardhan
Start Date: 8th Sep
Time: 11:00 AM

---
Course: Python
Trainer: Mr. Sai Vardhan
Start Date: 8th Sep
Time: 11:00 AM
```

```
---
Course: AWS Basics[For FS Java/Python]
Trainer: Mr. Tmnan
```

```
query = "Provide me Omkar sir upcoming class zoom link."
response = agent.run(f"{{system_instruction}}\n\nQuestion: {{query}}")

print("\n--- Admin Answer ---\n", response)
```



```
> Entering new AgentExecutor chain...
Thought: The student is asking for Omkar sir's upcoming class zoom link, I should first try to find this information in the document

Action:
...
{
  "action": "Document Retriever",
  "action_input": "Omkar sir upcoming class zoom link"
}
...

Observation: Welcome to Naresh IT. I'm happy to help you with the information you're Looking for. According to our records, Omkar Si
Thought:Thought: It seems Like the Document Retriever tool was able to find the relevant information about Omkar sir's upcoming clas

Final Answer: Hello! I've found the information you're Looking for. The Zoom Link for Omkar Sir's upcoming class is https://zoom.us/j/8653705807

> Finished chain.

--- Admin Answer ---
Hello! I've found the information you're looking for. The Zoom link for Omkar Sir's upcoming class is https://zoom.us/j/8653705807
```

```
# from langchain.agents import initialize_agent, Tool, AgentType
# from langchain.chains import RetrievalQA
# from bs4 import BeautifulSoup
# import requests

# course_schedule_url = "https://nareshit.in/course-schedule/"

# # 1Scraper tool
# def fetch_trainer_from_web(query: str):
#     """
#     Fetch ALL trainer/course details from NareshIT course-schedule page.
#     Input: trainer name or course name as plain text.
#     Returns: A student-friendly description string with all matches.
#     """
#     resp = requests.get(course_schedule_url, timeout=10)
#     if resp.status_code != 200:
#         return f"Could not reach schedule page (status code {resp.status_code})."

#     soup = BeautifulSoup(resp.text, "html.parser")
#     rows = soup.find_all("tr")

#     matches = []
#     for row in rows:
#         cols = [td.get_text(strip=True) for td in row.find_all("td")]
#         if any(query.lower() in c.lower() for c in cols):
#             course = cols[0] if len(cols)>0 else "Unknown"
#             trainer = cols[1] if len(cols)>1 else "Unknown"
#             date = cols[2] if len(cols)>2 else "Unknown"
#             time = cols[3] if len(cols)>3 else "Unknown"
#             matches.append(
#                 f"Course: {course}\n"
#                 f"Trainer: {trainer}\n"
#                 f"Start Date: {date}\n"
#                 f"Time: {time}\n"
#             )

#     if not matches:
#         return f"No matching course or trainer found for '{query}' on NareshIT schedule."
#     return "\n---\n".join(matches)

# trainer_tool = Tool(
#     name="Course Schedule Web Fetcher",
#     func=fetch_trainer_from_web,
#     description=(
#         "Use this tool to answer queries about courses, trainers, or session schedules "
#         "at Naresh IT. Input can be a course name (like 'Python') or trainer name."
#     )
# )
```

```
# # Optional document retriever (kept from your code)
# retriever_chain = RetrievalQA.from_chain_type(
#     llm=llm,
#     retriever=retriever,
#     chain_type="stuff"
# )
# retriever_tool = Tool(
#     name="Document Retriever",
#     func=lambda q: retriever_chain.run(q),
#     description="Answer queries using NareshIT training documents."
# )

# tools = [retriever_tool, trainer_tool]

# # 3Initialize agent with a more flexible agent type
# agent = initialize_agent(
#     tools=tools,
#     llm=llm,
#     agent=AgentType.CHAT_ZERO_SHOT_REACT_DESCRIPTION, # works with plain text tools
#     verbose=True
# )

# # System instruction
# system_instruction = """
# You are a helpful College Administrator at Naresh IT.
# - If the student asks about sessions, trainers, or course schedules, ALWAYS call the Course Schedule Web Fetcher tool.
# - If the student asks about institute profile, history, placement, or syllabus, use the Document Retriever.
# - Always give a friendly, student-facing response with final information.
# """

# # Query
# query = "Tell me about upcoming session on python."
# response = agent.run(f"{system_instruction}\n\nQuestion: {query}")

# print("\n--- Admin Answer ---\n", response)
```

✓ Spot ON-1

- Difference between RecursiveCharacterTextSplitter vs CharacterTextSplitter

1. Character-level splitting

Splits text into chunks of fixed character length (e.g., 500 characters per chunk).

- Pros:
 - Simple to implement.
- Cons:
 - Can cut sentences in half, losing semantic meaning.
 - Chunks might be arbitrary, making embeddings less semantically meaningful.

2. Recursive splitting (hierarchical)

Splits text intelligently, e.g.:

```
- First by document → sections
- Then paragraphs
- Then sentences, if needed.
```

- Pros:
 - Preserves semantic units (sentences/paragraphs).
 - Embeddings capture meaningful context, improving RAG performance.
- Cons:
 - Slightly more complex to implement.

```
# Example text
text = """
Artificial intelligence (AI) is the simulation of human intelligence in machines. These machines are programmed to think like humans and
Machine learning is a subset of AI that allows systems to learn and improve from experience without being explicitly programmed. Deep le
"""
```

```
# ----- 1. Character-level splitting -----
def split_by_char(text, chunk_size=100):
    return [text[i:i+chunk_size] for i in range(0, len(text), chunk_size)]

char_chunks = split_by_char(text, chunk_size=100)
print("Character-level chunks:\n")
for i, chunk in enumerate(char_chunks, 1):
    print(f"Chunk {i}:\n{chunk}\n")

# ----- 2. Recursive splitting (paragraph → sentence) -----
import re

def split_by_paragraph_and_sentence(text):
    paragraphs = [p.strip() for p in text.split("\n") if p.strip()]
    sentence_chunks = []
    for para in paragraphs:
        # Split paragraph into sentences
        sentences = re.split(r'(?<=[.!?])\s+', para)
        sentence_chunks.extend(sentences)
    return sentence_chunks

recursive_chunks = split_by_paragraph_and_sentence(text)
print("\nRecursive chunks (paragraph → sentence):\n")
for i, chunk in enumerate(recursive_chunks, 1):
    print(f"Chunk {i}:\n{chunk}\n")
```

↔ Character-level chunks:

- Chunk 1:
- Artificial intelligence (AI) is the simulation of human intelligence in machines. These machines are programmed to think like humans and mimic their actions. AI has applications in many industries, including healthcare, finance, and transportation.
- Chunk 2:
- Machine learning is a subset of AI that allows systems to learn and improve from experience without being explicitly programmed. Deep learning, a further subset of machine learning, uses neural networks to model complex patterns in data.
- Chunk 3:
- Artificial intelligence (AI) is the simulation of human intelligence in machines. These machines are programmed to think like humans and mimic their actions. AI has applications in many industries, including healthcare, finance, and transportation.
- Chunk 4:
- Machine learning is a subset of AI that allows systems to learn and improve from experience without being explicitly programmed. Deep learning, a further subset of machine learning, uses neural networks to model complex patterns in data.
- Chunk 5:
- Artificial intelligence (AI) is the simulation of human intelligence in machines. These machines are programmed to think like humans and mimic their actions. AI has applications in many industries, including healthcare, finance, and transportation.

↔ Recursive chunks (paragraph → sentence):

- Chunk 1:
- Artificial intelligence (AI) is the simulation of human intelligence in machines.
- Chunk 2:
- These machines are programmed to think like humans and mimic their actions.
- Chunk 3:
- AI has applications in many industries, including healthcare, finance, and transportation.
- Chunk 4:
- Machine learning is a subset of AI that allows systems to learn and improve from experience without being explicitly programmed.
- Chunk 5:
- Deep learning, a further subset of machine learning, uses neural networks to model complex patterns in data.

✓ SPOT-ON 2:

- Difference between VectorDBQA, RetrievalQA, and Multi-RetrievalQA

QA Type	Data Source(s)	Retrieval Method	Use Case / Notes
VectorDBQA	Vector database (e.g., Pinecone, FAISS, Chroma)	Vector similarity search	Best for large text corpora already embedded; retrieves semantically similar chunks
RetrievalQA	Single source (documents, PDFs, DBs)	Retriever queries source	Works with one knowledge base; simple setup for focused domains
Multi-RetrievalQA	Multiple sources (documents, PDFs, multiple DBs)	Multiple retrievers, aggregate results	Useful for heterogeneous knowledge bases; combines answers from several sources

✓ SPOT-ON 3:

Chain types

- stuff
- MAP Reduce
- Refine
- Map Rerank

Chain Type	How It Works	Pros 	
Stuff	All retrieved documents are “stuffed” into the prompt and sent to the LLM in one go.	Simple, fast, easy	Doesn’t scale if many do
Map Reduce	Map: LLM processes each doc individually → partial answers. Reduce: Combine partial answers into one final answer.	Scales to large docs, parallelizable	Slower, multiple LLM call
Refine	Start with first doc → LLM drafts answer → iteratively updates/refines answer with each new doc.	Produces detailed, context-aware answers	Sequential (slower), ans
Map Rerank	LLM evaluates each doc separately → scores/ranks relevance → selects best doc/answer.	Helps pick the <i>most relevant</i> answer, avoids noise	Doesn’t combine info fr

Stuff → “Throw everything at once.”

Map Reduce → “Summarize each piece, then combine summaries.”

Refine → “Keep improving the answer as more docs arrive.”

Map Rerank → “Pick the best single answer from many candidates.”